



MANUÁL

POUŽÍVATEĽSKÝ MANUÁL PROGRAMU MIPSIM

ABSTRAKT

Manuál k programu MIPSIM slúži na rýchle pochopenie jeho fungovania, vysvetlenie inštrukcií a jednotlivých funkcionalít.

Martin Gregor

Vedúci práce: Ing. Ján Hudec, PhD.

Obsah

Program	2
Fázy	2
Riešenie konfliktov	2
Základné funkcionality	3
Zápis	3
Pozorovanie	3
Kontrolný panel	3
Časová časť	4
Súborová časť	4
Informačná časť	4
Inštrukcie	5
Aritmetické inštrukcie	5
Bitové logické inštrukcie	5
Skokové inštrukcie	6
Pamäťové inštrukcie	6
Shiftovacie inštrukcie	6
Špeciálne inštrukcie	7

Program

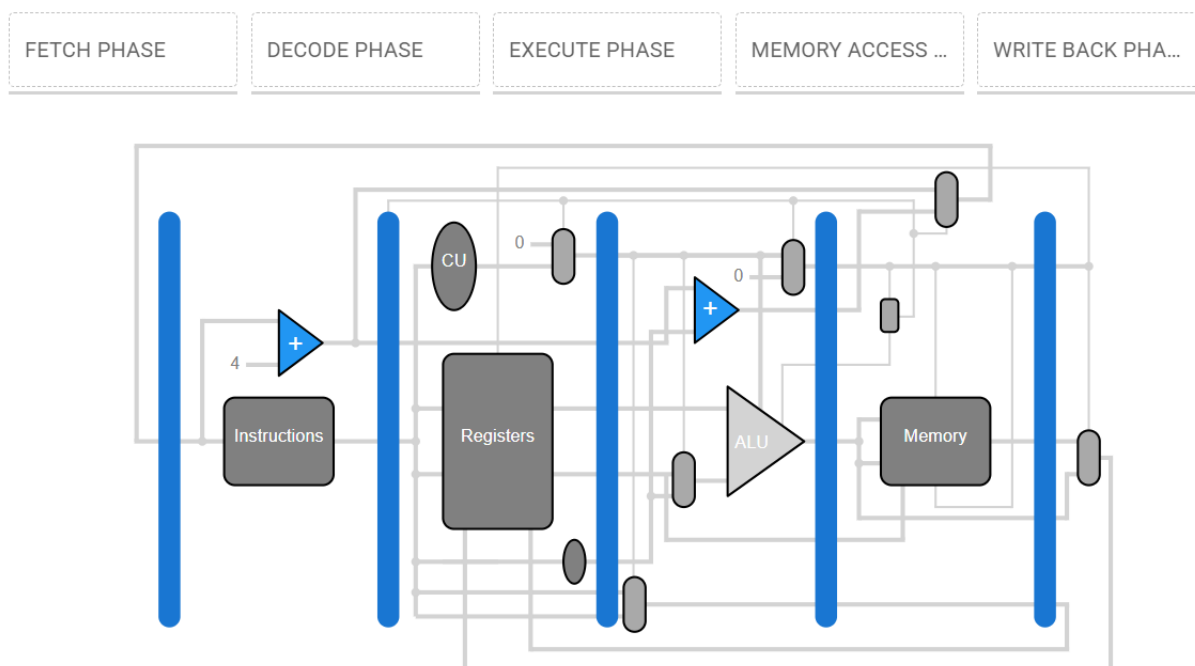
Program je vytvorený pomocou frameworku **Quasar** a **Vue**. Na správu stavov a prepojenie komponentov bola použitá knižnica **Pinia**. Na úpravu videa, ktoré je súčasťou programu, bol využitý nástroj **Clipchamp** spolu s platformou **Play.ht**, ktorá zabezpečila syntézu hlasu a tvorbu tituliek.

Fázy

V dokumentácii práce je podrobne popísaný každý krok vývoja programu, ako aj jeho celkové fungovanie. V stručnosti je program rozdelený do piatich fáz: **Fetch**, **Decode**, **Execute**, **Memory Access** a **Write Back**.

- **Fetch** – program načítava jednotlivé inštrukcie, aby vedel, čo má v ďalších fázach vykonať.
- **Decode** – inštrukcia je dekodovaná, pričom sa prípadne načítajú potrebné registre alebo údaje z pamäte.
- **Execute** – vykonávajú sa aritmetické alebo logické operácie pomocou **ALU**.
- **Memory Access** – zapisujú sa údaje späť do pamäte, ak to inštrukcia vyžaduje.
- **Write Back** – výsledky sú zapísané naspäť do registrov.

Tieto fázy prebiehajú paralelne, čo umožňuje **prúdové spracovanie inštrukcií (pipelining)**, čím sa zvyšuje efektívnosť vykonávania programu.



Riešenie konfliktov

Pri bežnom chode programu môže dôjsť ku konfliktu – zatiaľ čo jedna inštrukcia číta hodnotu z pamäte alebo registra, iná sa v tom istom momente pokúša túto hodnotu zapísať. Tento problém sa v niektorých programoch rieši vkladaním tzv. **NOP inštrukcií**, ktoré slúžia ako „pauza“ na odstránenie kolízie.

V našom programe však tento problém nenastáva. Hoci jednotlivé fázy spracovania inštrukcií prebiehajú simultánne (v rámci prúdového spracovania), nemajú rovnakú prioritu. **Priority sú nastavené od konca pipeline**, čo znamená, že fáza zápisu má prednosť pred čítaním.

Inými slovami: ak dôjde ku konfliktu, **najprv sa hodnota zapíše** do registra alebo pamäte a až následne sa umožní jej čítanie inou inštrukciou. Týmto spôsobom sa zabezpečuje konzistentnosť údajov bez potreby dodatočných zásahov do riadenia vykonávania.

To neznamena, že NOP-y nie je potrebné využívať vôbec naprieč fázami, ale vďaka prioritám ich počet môžeme znížiť.

Základné funkcionality

Program je možné rozdeliť do troch hlavných častí:

1. **Zápisová časť** - obsahuje registre, pamäť a zoznam inštrukcií. Užívateľ interaguje s jednotlivými poliami ktoré je schopné prepisovať aj počas samotného behu programu.
2. **Pozorovacia časť** - zahŕňa schému MIPSIM-u a vizualizáciu jednotlivých fáz, vrátane zobrazenia, v ktorej fáze sa aktuálne nachádza konkrétna inštrukcia.
3. **Kontrolný panel** – je ďalej rozdelený na tri podsekcie:
 - a. Časová časť
 - b. Súborová časť
 - c. Informačná časť

Zápis

Používateľ môže v ľubovoľnom kroku simulácie zapisovať do registrov, pamäte aj zoznamu inštrukcií. Na rozdiel od predchádzajúcich verzií MIPSIM-u, ak používateľ zadá nesprávnu syntax, použije neexistujúci register, návessie alebo adresu pamäte, program nespadne. **Simulácia pokračuje ďalej**, pričom sa zobrazí **chybové hlásenie** a chybná inštrukcia sa v internej logike programu **spracuje ako NOP** (žiadna operácia).

Týmto spôsobom je zabezpečené, že používateľ **nemusí manuálne prepisovať každú neplatnú alebo dočasne odstránenú inštrukciu na NOP** – stačí, ak ponechá pole prázdne alebo s chybným zápisom. Rovnako to platí aj pre registre a pamäť – ak používateľ odstráni hodnotu (napr. ponechá pole prázdne namiesto nuly), program nebude zlyhávať a simulácia bude pokračovať.

Je dôležité zdôrazniť, že v MIPSIM-e sú **pamäťové adresy rozdelené po 4 bajtoch**, teda prvá adresa je 0000, druhá 0004 atď. Pre zjednodušenie práce je však používateľovi zobrazovaná **reálna hodnota adresy** v ľavom hornom rohu pamäťovej tabuľky. **Používateľ pracuje so systémom v desiatkovej sústave**, čo je prirodzenejšie a prehľadnejšie. Napriek tomu má stále možnosť nahliadnuť, **ako sú dáta v skutočnosti uložené** v pamäti podľa reálnej adresácie MIPS architektúry.

Pozorovanie

V tejto časti je používateľ v pasívnej úlohe a má obmedzené možnosti interakcie so samotným prostredím. Slúži najmä ako vizuálna pomôcka na znázornenie toho, ako program prebieha – aké jednotky medzi sebou komunikujú a ako interagujú s pamäťou, registrami, inštrukčnou sadou, aritmeticko-logickou jednotkou...

Kontrolný panel

Pravdepodobne najdôležitejším prvkom celého programu je tzv. kontrolný panel. Slúži na manipuláciu s celým kódom a jeho následné transformovanie do požadovanej podoby. Ako už bolo spomenuté, **kontrolný panel je rozdelený do troch častí.**



Časová časť

Stredná časť kontrolného panela slúži na riadenie simulácie. Používateľ si v nej môže vybrať z **troch rôznych rýchlostí vykonávania programu**. Ak nechce, aby program bežal automaticky, má možnosť ho **manuálne krokovať** – teda posúvať sa inštrukciu po inštrukcii.

Program je možné **zastaviť** alebo **resetovať** v ľubovoľnom bode simulácie:

- **Zastavenie** – pozastaví chod programu, ale **zachová aktuálny stav všetkých komponentov** (inštrukcií, pamäte, registrov).
- **Resetovanie** – zachová obsah pamäte, registrov aj inštrukcií, ale **resetuje Program Counter**. Po opätovnom spustení tak program začne vykonávať inštrukcie od začiatku.

Používateľ má zároveň možnosť **samostatne vymazať obsah registrov alebo pamäte** – napríklad pri testovaní programu, keď potrebuje registre nastaviť späť na nulu alebo vyprázdniť pamäťové pole, do ktorého má program zapisovať.

Súborová časť

Pravá časť kontrolného panela slúži na prácu so súbormi. Používateľ má možnosť **importovať alebo exportovať** aktuálny stav inštrukcií, pamäte a registrov. Program pracuje so súbormi vo formáte **JSON**.

Odporúčame, aby používatelia **neupravovali tieto súbory mimo prostredia MIPSIM-u**, pokiaľ nie sú dostatočne skúsení. V prípade chybnnej manipulácie (napr. zmena počtu registrov alebo veľkosti pamäte) **nemusí byť program schopný súbor správne načítať**.

Súčasťou tejto časti je aj **tlačidlo na vytvorenie úplne nového programu**, ktoré **resetuje registre, pamäť, inštrukcie aj Program Counter** – čím sa v podstate spustí čisté pracovné prostredie.

Informačná časť

V **poslednej, ľavej časti rozhrania** sa nachádza **zoznam všetkých inštrukcií**. Používateľ si tu môže **prehliadnuť podrobný opis, vysvetlenie a syntax každej jednotlivé inštrukcie**.

Okrem toho je tu umiestnená aj **sekcia s nastaveniami**, v ktorej si používateľ môže zvoliť, **v akej číselnej sústave sa budú zobrazovať hodnoty** v registroch a pamäti – napríklad **desiatkovej, šestnástkovej alebo dvojkovej**. Ďalej je tu dostupná aj **možnosť zmeny jazyka rozhrania**, ktorá je síce pripravená na budúcu implementáciu (z dôvodu veľkosti knižnice), ale v aktuálnej verzii ešte nie je plne funkčná.

Poslednou súčasťou tejto časti je **nápoveda**, ktorá obsahuje:

- samotný **manuál k programu**,
- **videotutoriál** ako odľahčenú formu výučby, doplnený titulkami a komentárom,
- a **stručný opis programu**, ktorý pomáha novým používateľom rýchlo sa zorientovať.

Inštrukcie

Inštrukcie sú detailne popísané aj priamo v programe. Pre lepší prehľad a ucelenie sú však uvedené aj v tomto dokumente – pre tých, ktorým vyhovuje čítanie inštrukcií vo formáte PDF skôr než priamo v aplikácii.

Inštrukcie môžeme rozdeliť na aritmetické, bitové, skokové, pamäťové, shiftovacie a špeciálne.

Aritmetické inštrukcie

Inštrukcia	Syntax	Opis
ADD (Addition)	ADD \$Rx \$Ry \$Rz	Inštrukcia spočíta registre Ry a Rz. Výsledok uloží do registra Rx.
ADDI (Addition Immediate)	ADDI \$Rx \$Ry \$i	Inštrukcia spočíta register Ry a konštantu i. Výsledok uloží do registra Rx.
SUB (Subtraction)	SUB \$Rx \$Ry \$Rz	Inštrukcia odčíta od registra Ry register Rz. Výsledok uloží do registra Rx.
SUBI (Subtraction Immediate)	SUBI \$Rx \$Ry \$i	Inštrukcia odčíta od registra Ry konštantu i. Výsledok uloží do registra Rx.
MUL (Multiply)	MUL \$Rx \$Ry \$Rz	Inštrukcia vynásobí registre Ry a Rz. Výsledok uloží do registra Rx.
MULI (Multiply Immediate)	MULI \$Rx \$Ry \$i	Inštrukcia vynásobí register Ry a konštantu i. Výsledok uloží do registra Rx.
MULU (Multiply Unsigned)	MULU \$Rx \$Ry \$Rz	Inštrukcia vynásobí registre Ry a Rz. Výsledok uloží do registra Rx. (Unsigned)
DIV (Divide)	DIV \$Rx \$Ry \$Rz	Inštrukcia vydolí register Ry registrom Rz. Výsledok uloží do registra Rx.
DIVI (Divide Immediate)	DIVI \$Rx \$Ry \$i	Inštrukcia vydolí register Ry konštantou i. Výsledok uloží do registra Rx.
DIVU (Divide Unsigned)	DIVU \$Rx \$Ry \$Rz	Inštrukcia vydolí register Ry registrom Rz. Výsledok uloží do registra Rx. (Unsigned)

Bitové logické inštrukcie

Inštrukcia	Syntax	Opis
AND (Bitwise AND)	AND \$Rx \$Ry \$Rz	Bitový AND medzi registrami Ry a Rz. Výsledok uloží do registra Rx.
ANDI (Bitwise AND Immediate)	ANDI \$Rx \$Ry \$i	Bitový AND medzi registrom Ry a konštantou i. Výsledok uloží do registra Rx.
OR (Bitwise OR)	OR \$Rx \$Ry \$Rz	Bitový OR medzi registrami Ry a Rz. Výsledok uloží do registra Rx.
ORI (Bitwise OR Immediate)	ORI \$Rx \$Ry \$i	Bitový OR medzi registrom Ry a konštantou i. Výsledok uloží do registra Rx.
XOR (Bitwise XOR)	XOR \$Rx \$Ry \$Rz	Bitový XOR medzi registrami Ry a Rz. Výsledok uloží do registra Rx.
XORI (Bitwise XOR Immediate)	XORI \$Rx \$Ry \$i	Bitový XOR medzi registrom Ry a konštantou i. Výsledok uloží do registra Rx.
NAND (Bitwise NAND)	NAND \$Rx \$Ry \$Rz	Bitový negovaný AND medzi registrami Ry a Rz. Výsledok uloží do registra Rx.

NANDI (Bitwise NAND Immediate)	NANDI \$Rx \$Ry \$i	Bitový negovaný AND medzi registrom Ry a konštantou i. Výsledok uloží do registra Rx.
NOR (Bitwise NOR)	NOR \$Rx \$Ry \$Rz	Bitový negovaný OR medzi registrami Ry a Rz. Výsledok uloží do registra Rx.
NORI (Bitwise NOR Immediate)	NORI \$Rx \$Ry \$i	Bitový negovaný OR medzi registrom Ry a konštantou i. Výsledok uloží do registra Rx.
XNOR (Bitwise XNOR)	XNOR \$Rx \$Ry \$Rz	Bitový negovaný XOR medzi registrami Ry a Rz. Výsledok uloží do registra Rx.
XNORI (Bitwise XNOR Immediate)	XNORI \$Rx \$Ry \$i	Bitový negovaný XOR medzi registrom Ry a konštantou i. Výsledok uloží do registra Rx.

Skokové inštrukcie

Inštrukcia	Syntax	Opis
BEQ (Branch if Equal)	BEQ \$Rx \$Ry \$instruction	Inštrukcia vykoná skok na návestie, ak sa registre Rx a Ry rovnajú.
BNEQ (Branch if Not Equal)	BNEQ \$Rx \$Ry \$instruction	Inštrukcia vykoná skok na návestie, ak sa registre Rx a Ry nerovnajú.
BHI (Branch if Higher)	BHI \$Rx \$Ry \$instruction	Inštrukcia vykoná skok na návestie, ak je register Rx väčší ako register Ry.
BLO (Branch if Lower)	BLO \$Rx \$Ry \$instruction	Inštrukcia vykoná skok na návestie, ak je register Rx menší ako register Ry.
J (JUMP)	J \$instruction	Inštrukcia vykoná nepodmienенý skok na návestie.

Pamäťové inštrukcie

Inštrukcia	Syntax	Opis
LW (Load Word)	LW \$Rx \$Ry \$memory	Inštrukcia načíta slovo z pamäte na adrese určenej súčtom hodnoty v registri Ry a offsetu. Načítané slovo uloží do registra Rx.
LWI (Load Word Immediate)	LWI \$Rx \$i \$memory	Inštrukcia načíta slovo z pamäte na adrese určenej súčtom hodnoty konštanty i a offsetu. Načítané slovo uloží do registra Rx.
SW (Store Word)	SW \$Rx \$Ry \$memory	Inštrukcia uloží slovo do pamäte na adresu určenej súčtom hodnoty v registri Ry a offsetu. Uložené slovo je z registra Rx.
SWI (Store Word Immediate)	SWI \$Rx \$i \$memory	Inštrukcia uloží slovo do pamäte na adresu určenej súčtom hodnoty konštanty i a offsetu. Uložené slovo je z registra Rx.

Shiftovacie inštrukcie

Inštrukcia	Syntax	Opis
SLLV (Shift Left Logical Variable)	SLLV \$Rx \$Ry \$Rz	Inštrukcia posunie bitovú hodnotu čísla v registri Ry o počet pozícií určený hodnotou v registri Rz doľava. Výsledok uloží do registra Rx.

SRLV (Shift Right Logical Variable)	SRLV \$Rx \$Ry \$Rz	Inštrukcia posunie bitovú hodnotu čísla v registri Ry o počet pozícií určený hodnotou v registri Rz doprava. Výsledok uloží do registra Rx.
SLLVI (Shift Left Logical Variable Immediate)	SLLVI \$Rx \$Ry \$i	Inštrukcia posunie bitovú hodnotu čísla v registri Ry o počet pozícií určený hodnotou konštanty i doľava. Výsledok uloží do registra Rx.
SRLVI (Shift Right Logical Variable Immediate)	SRLVI \$Rx \$Ry \$i	Inštrukcia posunie bitovú hodnotu čísla v registri Ry o počet pozícií určený hodnotou konštanty i doprava. Výsledok uloží do registra Rx.

Špeciálne inštrukcie

Inštrukcia	Syntax	Opis
NOP (No Operation)	NOP	Inštrukcia ktorá nevykoná žiadnu operáciu.
LI (Load Immediate)	LI \$Rx \$i	Inštrukcia načíta konštantu i a uloží ju do registra Rx.
LUI (Load Upper Immediate)	LUI \$Rx \$i	Inštrukcia načíta vrchných 16 bitov konštanty i a uloží ju do registra Rx.
Q (QUIT)	Q	Predčasné ukončenie programu.